RFQ Automation Platform - Backend Project Structure

Project Overview

Technology Stack:

- **Runtime:** Node.js 18+
- **Framework:** Express.js with TypeScript
- **Database:** PostgreSQL with Prisma ORM
- **Authentication:** JWT with refresh tokens
- **Queue System:** Bull (Redis-based)
- **Email Service:** Nodemailer with SendGrid
- **File Storage:** AWS S3 or local filesystem
- **Testing:** Jest with Supertest
- **Documentation:** Swagger/OpenAPI

Directory Structure

```
rfq-automation-backend/
 ├── src/
|   ├── config/          # Configuration files
|   ├── controllers/     # Request handlers
|   ├── middleware/      # Express middleware
|   ├── models/          # Database models & schemas
|   ├── routes/          # API route definitions
|   ├── services/        # Business logic layer
|   ├── utils/           # Helper functions
|   ├── types/           # TypeScript type definitions
|   ├── validators/      # Request validation schemas
|   ├── jobs/            # Background job processors
|   └── app.ts           # Express app setup
```

```
├── prisma/
│   ├── schema.prisma    # Database schema
│   ├── migrations/      # Database migrations
│   └── seed.ts          # Seed data
├── tests/
│   ├── unit/            # Unit tests
│   ├── integration/     # Integration tests
│   ├── fixtures/        # Test data
│   └── helpers/         # Test utilities
├── docs/
│   ├── api/             # API documentation
│   └── deployment/      # Deployment guides
├── scripts/
│   ├── build.sh         # Build scripts
│   ├── migrate.sh       # Database migration
│   └── deploy.sh        # Deployment script
├── docker/
│   ├── Dockerfile
│   ├── docker-compose.yml
│   └── docker-compose.dev.yml
├── .env.example         # Environment variables template
├── package.json         # Dependencies and scripts
├── tsconfig.json        # TypeScript configuration
├── jest.config.js       # Test configuration
└── README.md            # Project documentation
```

Core Configuration Files

package.json

```
{
  "name": "rfq-automation-backend",
  "version": "1.0.0",
  "description": "RFQ Automation Platform API",
  "main": "dist/app.js",
  "scripts": {
    "dev": "nodemon src/app.ts",
    "build": "tsc && tsc-alias",
    "start": "node dist/app.js",
    "test": "jest",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage",
    "migrate": "prisma migrate dev",
    "migrate:prod": "prisma migrate deploy",
    "seed": "ts-node prisma/seed.ts",
    "lint": "eslint src/**/*.ts",
    "lint:fix": "eslint src/**/*.ts --fix"
  },
  "dependencies": {
    "express": "^4.18.2",
    "cors": "^2.8.5",
    "helmet": "^7.0.0",
    "compression": "^1.7.4",
    "express-rate-limit": "^6.8.1",
    "express-validator": "^7.0.1",
    "@prisma/client": "^5.0.0",
    "jsonwebtoken": "^9.0.1",
```

```
    "bcryptjs": "^2.4.3",
    "nodemailer": "^6.9.3",
    "@sendgrid/mail": "^7.7.0",
    "bull": "^4.10.4",
    "ioredis": "^5.3.2",
    "multer": "^1.4.5-lts.1",
    "aws-sdk": "^2.1400.0",
    "stripe": "^12.10.0",
    "winston": "^3.9.0",
    "dotenv": "^16.3.1",
    "swagger-jsdoc": "^6.2.8",
    "swagger-ui-express": "^4.6.3",
    "openai": "^3.3.0"
  },
  "devDependencies": {
    "@types/express": "^4.17.17",
    "@types/node": "^20.3.1",
    "@types/cors": "^2.8.13",
    "@types/compression": "^1.7.2",
    "@types/bcryptjs": "^2.4.2",
    "@types/jsonwebtoken": "^9.0.2",
    "@types/nodemailer": "^6.4.8",
    "@types/multer": "^1.4.7",
    "@types/jest": "^29.5.2",
    "@types/supertest": "^2.0.12",
    "typescript": "^5.1.3",
    "ts-node": "^10.9.1",
    "tsc-alias": "^1.8.6",
```

```
    "nodemon": "^2.0.22",
    "prisma": "^5.0.0",
    "jest": "^29.5.0",
    "ts-jest": "^29.1.0",
    "supertest": "^6.3.3",
    "@typescript-eslint/eslint-plugin": "^5.59.11",
    "@typescript-eslint/parser": "^5.59.11",
    "eslint": "^8.42.0"
  }
}
```

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "lib": ["ES2020"],
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true,
    "experimentalDecorators": true,
    "emitDecoratorMetadata": true,
    "baseUrl": "./src",
    "paths": {
      "@config/*": ["config/*"],
```

```
    "@controllers/*": ["controllers/*"],
    "@middleware/*": ["middleware/*"],
    "@models/*": ["models/*"],
    "@routes/*": ["routes/*"],
    "@services/*": ["services/*"],
    "@utils/*": ["utils/*"],
    "@types/*": ["types/*"],
    "@validators/*": ["validators/*"],
    "@jobs/*": ["jobs/*"]
  }
 },
 "include": ["src/**/*"],
 "exclude": ["node_modules", "dist", "tests"]
}
```

.env.example

```
# Server Configuration
NODE_ENV=development
PORT=3000
API_VERSION=v1

# Database
DATABASE_URL="postgresql://username:password@localhost:5432/rfq_platform"
DATABASE_POOL_MIN=5
DATABASE_POOL_MAX=100

# Redis (for queues and caching)
REDIS_URL="redis://localhost:6379"
REDIS_PASSWORD=
```

```
REDIS_DB=0

# JWT Authentication
JWT_SECRET=your-super-secret-jwt-key
JWT_EXPIRES_IN=15m
REFRESH_TOKEN_SECRET=your-refresh-token-secret
REFRESH_TOKEN_EXPIRES_IN=7d

# Email Configuration
EMAIL_PROVIDER=sendgrid
SENDGRID_API_KEY=your-sendgrid-api-key
EMAIL_FROM=noreply@yourcompany.com
EMAIL_FROM_NAME="RFQ Platform"

# SMTP (alternative to SendGrid)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your-email@gmail.com
SMTP_PASSWORD=your-app-password

# File Storage
STORAGE_PROVIDER=aws
AWS_REGION=us-east-1
AWS_ACCESS_KEY_ID=your-aws-access-key
AWS_SECRET_ACCESS_KEY=your-aws-secret
AWS_S3_BUCKET=rfq-platform-files

# Local storage (alternative to AWS)
```

```
UPLOAD_DIR=uploads
MAX_FILE_SIZE=10485760

# Stripe (for billing)
STRIPE_SECRET_KEY=sk_test_your-stripe-secret
STRIPE_WEBHOOK_SECRET=whsec_your-webhook-secret

# OpenAI (for AI quote parsing)
OPENAI_API_KEY=sk-your-openai-key
OPENAI_MODEL=gpt-4

# Rate Limiting
RATE_LIMIT_WINDOW_MS=900000
RATE_LIMIT_MAX_REQUESTS=1000

# CORS
CORS_ORIGINS=http://localhost:3001,https://yourdomain.com

# Logging
LOG_LEVEL=info
LOG_FILE=logs/app.log

# Feature Flags
ENABLE_AI_PARSING=true
ENABLE_WEBHOOKS=true
ENABLE_ANALYTICS=true
```

Application Structure

src/app.ts - Main Application Entry

```
import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import compression from 'compression';
import rateLimit from 'express-rate-limit';
import swaggerUi from 'swagger-ui-express';
import swaggerSpec from '@config/swagger';
import { errorHandler } from '@middleware/errorHandler';
import { requestLogger } from '@middleware/logger';
import { corsOptions } from '@config/cors';
import { rateLimitConfig } from '@config/rateLimit';
import routes from '@routes/index';
import { connectDatabase } from '@config/database';
import { initializeQueues } from '@config/queues';
import logger from '@utils/logger';

const app = express();

// Security middleware
app.use(helmet());
app.use(cors(corsOptions));
app.use(compression());
app.use(rateLimit(rateLimitConfig));

// Body parsing middleware
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true, limit: '10mb' }));
```

```
// Logging middleware
app.use(requestLogger);

// API Documentation
app.use('/api/docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));

// API Routes
app.use('/api/v1', routes);

// Health check endpoint
app.get('/health', (req, res) => {
  res.status(200).json({
    status: 'OK',
    timestamp: new Date().toISOString(),
    uptime: process.uptime(),
    version: process.env.npm_package_version
  });
});

// Error handling middleware (must be last)
app.use(errorHandler);

// Initialize services and start server
const PORT = process.env.PORT || 3000;

async function startServer() {
  try {
```

```
    // Connect to database
    await connectDatabase();

    // Initialize background job queues
    await initializeQueues();

    app.listen(PORT, () => {
      logger.info(`🚀 Server running on port ${PORT}`);
      logger.info(`📚 API Documentation: http://localhost:${PORT}/api/docs`);
    });
  } catch (error) {
    logger.error('Failed to start server:', error);
    process.exit(1);
  }
}

startServer();

export default app;
```

Configuration Layer

src/config/database.ts

```
import { PrismaClient } from '@prisma/client';
import logger from '@utils/logger';

let prisma: PrismaClient;

export const connectDatabase = async () => {
```

```
  try {
    prisma = new PrismaClient({
      log: [
        { emit: 'event', level: 'query' },
        { emit: 'event', level: 'error' },
        { emit: 'event', level: 'warn' }
      ]
    });

    // Log slow queries in development
    if (process.env.NODE_ENV === 'development') {
      prisma.$on('query', (e) => {
        if (e.duration > 1000) {
          logger.warn(`Slow query detected: ${e.duration}ms - ${e.query}`);
        }
      });
    }

    await prisma.$connect();
    logger.info('✅ Database connected successfully');
  } catch (error) {
    logger.error('❌ Database connection failed:', error);
    throw error;
  }
};

export const disconnectDatabase = async () => {
  if (prisma) {
```

```
    await prisma.$disconnect();
    logger.info('Database disconnected');
  }
};

export { prisma };
```

src/config/queues.ts

```
import Queue from 'bull';
import Redis from 'ioredis';
import logger from '@utils/logger';

const redis = new Redis(process.env.REDIS_URL!);

// Queue definitions
export const emailQueue = new Queue('email processing', {
  redis: {
    port: parseInt(process.env.REDIS_PORT || '6379'),
    host: process.env.REDIS_HOST || 'localhost',
    password: process.env.REDIS_PASSWORD
  }
});

export const aiParsingQueue = new Queue('AI quote parsing', {
  redis: {
    port: parseInt(process.env.REDIS_PORT || '6379'),
    host: process.env.REDIS_HOST || 'localhost',
    password: process.env.REDIS_PASSWORD
  }
```

```
});

export const analyticsQueue = new Queue('analytics processing', {
  redis: {
    port: parseInt(process.env.REDIS_PORT || '6379'),
    host: process.env.REDIS_HOST || 'localhost',
    password: process.env.REDIS_PASSWORD
  }
});

export const initializeQueues = async () => {
  try {
    // Import job processors
    const { processEmailJobs } = await import('@jobs/emailProcessor');
    const { processAIParsingJobs } = await import('@jobs/aiParsingProcessor');
    const { processAnalyticsJobs } = await import('@jobs/analyticsProcessor');

    // Register job processors
    emailQueue.process('send-email', 5, processEmailJobs);
    emailQueue.process('send-bulk-emails', 3, processEmailJobs);

    aiParsingQueue.process('parse-quote-email', 2, processAIParsingJobs);

    analyticsQueue.process('calculate-metrics', 1, processAnalyticsJobs);

    // Queue event listeners
    emailQueue.on('completed', (job) => {
      logger.info(`Email job ${job.id} completed`);
```

```
  });

  emailQueue.on('failed', (job, err) => {
    logger.error(`Email job ${job.id} failed: ${err.message}`);
  });

  logger.info('✅ Background queues initialized');
 } catch (error) {
  logger.error('❌ Queue initialization failed:', error);
  throw error;
 }
};
```

Middleware Layer

src/middleware/auth.ts

```
import { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';
import { prisma } from '@config/database';
import { AppError } from '@utils/errors';
import { UserRole } from '@types/auth';

interface JWTPayload {
 userId: string;
 companyId?: string;
 userType: 'admin' | 'company_user';
 role: UserRole;
 iat: number;
 exp: number;
```

```
}

export interface AuthenticatedRequest extends Request {
  user: {
    id: string;
    companyId?: string;
    userType: 'admin' | 'company_user';
    role: UserRole;
  };
}

export const authenticate = async (
  req: Request,
  res: Response,
  next: NextFunction
) => {
  try {
    const token = extractToken(req);

    if (!token) {
      throw new AppError('Authentication token required', 401);
    }

    const payload = jwt.verify(token, process.env.JWT_SECRET!) as JWTPayload;

    // Verify user still exists and is active
    let user;
    if (payload.userType === 'admin') {
```

```
    user = await prisma.admin.findFirst({
      where: { id: payload.userId, isActive: true }
    });
  } else {
    user = await prisma.companyUser.findFirst({
      where: {
        id: payload.userId,
        isActive: true,
        company: { isActive: true }
      },
      include: { company: true }
    });
  }

  if (!user) {
    throw new AppError('User not found or inactive', 401);
  }

  (req as AuthenticatedRequest).user = {
    id: payload.userId,
    companyId: payload.companyId,
    userType: payload.userType,
    role: payload.role
  };

  next();
} catch (error) {
  if (error instanceof jwt.JsonWebTokenError) {
```

```
      next(new AppError('Invalid authentication token', 401));
    } else {
      next(error);
    }
  }
};

export const requireRole = (roles: UserRole[]) => {
  return (req: Request, res: Response, next: NextFunction) => {
    const authReq = req as AuthenticatedRequest;

    if (!roles.includes(authReq.user.role)) {
      throw new AppError('Insufficient permissions', 403);
    }

    next();
  };
};

export const requireAdmin = (req: Request, res: Response, next: NextFunction) => {
  const authReq = req as AuthenticatedRequest;

  if (authReq.user.userType !== 'admin') {
    throw new AppError('Admin access required', 403);
  }

  next();
};
```

```
const extractToken = (req: Request): string | null => {
  const authHeader = req.headers.authorization;

  if (authHeader && authHeader.startsWith('Bearer ')) {
    return authHeader.substring(7);
  }

  return null;
};
```

src/middleware/tenantIsolation.ts

```
import { Response, NextFunction } from 'express';
import { AuthenticatedRequest } from './auth';
import { AppError } from '@utils/errors';

export const enforceTenantIsolation = (
  req: AuthenticatedRequest,
  res: Response,
  next: NextFunction
) => {
  // Skip tenant isolation for platform admins
  if (req.user.userType === 'admin') {
    return next();
  }

  // Ensure company user has companyId
  if (!req.user.companyId) {
    throw new AppError('Company context required', 400);
```

```
  }

  // Add tenant context to query parameters if not present
  if (req.method === 'GET' || req.method === 'POST' || req.method === 'PUT') {
    req.query.companyId = req.user.companyId;

    if (req.body && typeof req.body === 'object') {
      req.body.companyId = req.user.companyId;
    }
  }

  next();
};

export const validateTenantAccess = (resourceCompanyId: string, userCompanyId?:
string) => {
  if (!userCompanyId) {
    throw new AppError('Company context required', 400);
  }

  if (resourceCompanyId !== userCompanyId) {
    throw new AppError('Access denied to resource', 403);
  }
};
```

Service Layer

src/services/RFQService.ts

```
import { prisma } from '@config/database';
```

```
import { emailQueue } from '@config/queues';
import { AppError } from '@utils/errors';
import { validateTenantAccess } from '@middleware/tenantIsolation';
import { RFQStatus, RFQCreateData, RFQUpdateData } from '@types/rfq';
import { generateRFQNumber } from '@utils/generators';
import logger from '@utils/logger';

export class RFQService {
  async createRFQ(data: RFQCreateData, userId: string, companyId: string) {
    try {
      const rfqNumber = await generateRFQNumber(companyId);

      const rfq = await prisma.rfq.create({
        data: {
          ...data,
          rfqNumber,
          companyId,
          createdBy: userId,
          status: RFQStatus.DRAFT
        },
        include: {
          createdByUser: true,
          company: true
        }
      });

      logger.info(`RFQ created: ${rfq.rfqNumber}`, { rfqId: rfq.id, companyId });
      return rfq;
```

```
    } catch (error) {
      logger.error('Failed to create RFQ:', error);
      throw new AppError('Failed to create RFQ', 500);
    }
  }

  async updateRFQ(id: string, data: RFQUpdateData, userId: string, companyId: string)
{
    // Verify RFQ exists and belongs to company
    const existingRFQ = await this.getRFQById(id, companyId);

    if (existingRFQ.status !== RFQStatus.DRAFT) {
      throw new AppError('Cannot edit sent RFQ', 400);
    }

    try {
      const rfq = await prisma.rfq.update({
        where: { id },
        data: {
          ...data,
          updatedAt: new Date()
        },
        include: {
          createdByUser: true,
          quotes: true
        }
      });
```

```
    logger.info(`RFQ updated: ${rfq.rfqNumber}`, { rfqId: rfq.id });
    return rfq;
  } catch (error) {
    logger.error('Failed to update RFQ:', error);
    throw new AppError('Failed to update RFQ', 500);
  }
}

async sendRFQ(id: string, contactIds: string[], companyId: string) {
  const rfq = await this.getRFQById(id, companyId);

  if (rfq.status !== RFQStatus.DRAFT) {
    throw new AppError('RFQ already sent', 400);
  }

  // Validate contacts belong to company
  const contacts = await prisma.contact.findMany({
    where: {
      id: { in: contactIds },
      companyId,
      isActive: true,
      doNotContact: false
    },
    include: {
      shippingLine: true
    }
  });
```

```
if (contacts.length !== contactIds.length) {
  throw new AppError('Invalid or inactive contacts provided', 400);
}

const transaction = await prisma.$transaction(async (prisma) => {
  // Update RFQ status
  const updatedRFQ = await prisma.rfq.update({
    where: { id },
    data: {
      status: RFQStatus.SENT,
      sentAt: new Date(),
      totalContactsSent: contacts.length
    }
  });

  // Create recipient records
  const recipients = await Promise.all(
    contacts.map(contact =>
      prisma.rfqRecipient.create({
        data: {
          rfqId: id,
          contactId: contact.id,
          shippingLineId: contact.shippingLineId,
          emailSentAt: new Date()
        }
      })
    )
  );
```

```
  return { rfq: updatedRFQ, recipients };
});

// Queue email sending jobs
await Promise.all(
  contacts.map(contact =>
    emailQueue.add('send-email', {
      type: 'rfq',
      rfqId: id,
      contactId: contact.id,
      templateData: {
        contactName: `${contact.firstName} ${contact.lastName}`,
        companyName: rfq.company.name,
        rfqNumber: rfq.rfqNumber,
        rfqDetails: {
          title: rfq.title,
          origin: rfq.originPort,
          destination: rfq.destinationPort,
          containerType: rfq.containerType,
          quantity: rfq.containerQuantity
        }
      }
    })
  )
);

logger.info(`RFQ sent to ${contacts.length} contacts`, {
```

```
    rfqId: id,
    rfqNumber: rfq.rfqNumber
  });

  return transaction.rfq;
}

async getRFQById(id: string, companyId?: string) {
  const rfq = await prisma.rfq.findFirst({
    where: { id },
    include: {
      company: true,
      createdByUser: true,
      assignedToUser: true,
      recipients: {
        include: {
          contact: true,
          shippingLine: true
        }
      },
      quotes: {
        include: {
          contact: true,
          shippingLine: true
        }
      }
    }
  });
```

```
  if (!rfq) {
    throw new AppError('RFQ not found', 404);
  }

  // Validate tenant access for company users
  if (companyId) {
    validateTenantAccess(rfq.companyId, companyId);
  }

  return rfq;
}

async listRFQs(companyId: string, filters: {
  status?: RFQStatus;
  page?: number;
  limit?: number;
  search?: string;
}) {
  const { status, page = 1, limit = 20, search } = filters;
  const skip = (page - 1) * limit;

  const where = {
    companyId,
    ...(status && { status }),
    ...(search && {
      OR: [
        { title: { contains: search, mode: 'insensitive' as const } },
```

```
      { rfqNumber: { contains: search, mode: 'insensitive' as const } },
      { originPort: { contains: search, mode: 'insensitive' as const } },
      { destinationPort: { contains: search, mode: 'insensitive' as const } }
    ]
  })
};

const [rfqs, total] = await Promise.all([
  prisma.rfq.findMany({
    where,
    skip,
    take: limit,
    orderBy: { createdAt: 'desc' },
    include: {
      createdByUser: {
        select: { id: true, firstName: true, lastName: true }
      },
      _count: {
        select: { quotes: true, recipients: true }
      }
    }
  }),
  prisma.rfq.count({ where })
]);

return {
  rfqs,
  pagination: {
```

```
      current: page,
      pages: Math.ceil(total / limit),
      total,
      limit
    }
  };
 }
}
```

Controller Layer

src/controllers/RFQController.ts

```
import { Response, NextFunction } from 'express';
import { AuthenticatedRequest } from '@middleware/auth';
import { RFQService } from '@services/RFQService';
import { AppError } from '@utils/errors';
import { successResponse } from '@utils/response';
import logger from '@utils/logger';

const rfqService = new RFQService();

export class RFQController {
 async createRFQ(req: AuthenticatedRequest, res: Response, next: NextFunction) {
  try {
    const rfq = await rfqService.createRFQ(
      req.body,
      req.user.id,
      req.user.companyId!
    );
```

```
    return successResponse(res, rfq, 'RFQ created successfully', 201);
  } catch (error) {
    next(error);
  }
}

async updateRFQ(req: AuthenticatedRequest, res: Response, next: NextFunction) {
  try {
    const { id } = req.params;

    const rfq = await rfqService.updateRFQ(
      id,
      req.body,
      req.user.id,
      req.user.companyId!
    );

    return successResponse(res, rfq, 'RFQ updated successfully');
  } catch (error) {
    next(error);
  }
}

async sendRFQ(req: AuthenticatedRequest, res: Response, next: NextFunction) {
  try {
    const { id } = req.params;
    const { contactIds } = req.body;
```

```
    if (!Array.isArray(contactIds) || contactIds.length === 0) {
      throw new AppError('At least one contact must be selected', 400);
    }

    const rfq = await rfqService.sendRFQ(id, contactIds, req.user.companyId!);

    return successResponse(res, rfq, 'RFQ sent successfully');
  } catch (error) {
    next(error);
  }
}

async getRFQ(req: AuthenticatedRequest, res: Response, next: NextFunction) {
  try {
    const { id } = req.params;

    const rfq = await rfqService.getRFQById(
      id,
      req.user.userType === 'company_user' ? req.user.companyId : undefined
    );

    return successResponse(res, rfq, 'RFQ retrieved successfully');
  } catch (error) {
    next(error);
  }
}
```

```
async listRFQs(req: AuthenticatedRequest, res: Response, next: NextFunction) {
  try {
    const { status, page, limit, search } = req.query;

    const result = await rfqService.listRFQs(req.user.companyId!, {
      status: status as any,
      page: page ? parseInt(page as string) : undefined,
      limit: limit ? parseInt(limit as string) : undefined,
      search: search as string
    });

    return successResponse(res, result, 'RFQs retrieved successfully');
  } catch (error) {
    next(error);
  }
}

async closeRFQ(req: AuthenticatedRequest, res: Response, next: NextFunction) {
  try {
    const { id } = req.params;

    const rfq = await rfqService.updateRFQ(
      id,
      { status: 'closed', closedAt: new Date() },
      req.user.id,
      req.user.companyId!
    );
```

```
    return successResponse(res, rfq, 'RFQ closed successfully');
  } catch (error) {
    next(error);
  }
 }
}
```

Route Definitions

src/routes/index.ts

```
import express from 'express';
import authRoutes from './auth';
import adminRoutes from './admin';
import rfqRoutes from './rfq';
import quoteRoutes from './quote';
import contactRoutes from './contact';
import shippingLineRoutes from './shippingLine';
import emailRoutes from './email';
import analyticsRoutes from './analytics';
import webhookRoutes from './webhook';

const router = express.Router();

// Public routes (no authentication required)
router.use('/auth', authRoutes);
router.use('/webhooks', webhookRoutes);

// Protected routes (authentication required)
router.use('/admin', adminRoutes);
```

```
router.use('/rfqs', rfqRoutes);

router.use('/quotes', quoteRou
```